

One-command OpenID conformance testing for MOSIP Inji Certify & Inji Verify

Team FSD • Sudhanshu (team lead) • IIT-Bangalore, Synergy '26

Track: Digital Credential Trust Infrastructure

Do ck er	Pyt hon / htt px	Java - TestNG - Extent	GitH ub Acti ons	OI DF Su ite	Ope nID 4VC I 1.0	Ope nID4 VP / HAI P
----------------	------------------------------	------------------------------	---------------------------	-----------------------	----------------------------	---------------------------------

What this document is

This PDF is the pitch + evidence companion to the AutoConform repository (Testing_for_inji_certify_and_inji-verify-main). It is organised to mirror the official **MOSIP Decode 2026 Submission Guidelines** - every mandatory item (source code, working prototype, pitch deck, installation instructions, git history) plus every optional artefact (architecture diagram, test data, test cases & results, API documentation) has a dedicated, cross-referenced section. Section numbers in brackets, e.g. [G7], map to the checklist on the final page.

1 command full conformance run end-to-end	7 containers suite + both Inji products	28 modules 9 issuer + 2 verifier variants	18 known gaps documented with root causes	0 humans in the WAITING handoff loop
--	---	--	--	---

Contents

1	Executive summary	the pitch in 30 seconds
2	The problem	why conformance testing hurts today
3	The solution	what we built
4	Architecture	system design and data flow
5	Technology stack	and why each layer was chosen
6	Key features	orchestration engine / WAITING handoff / results governance
7	Working prototype & demo	[G2] what a live run looks like
8	Test data	[G8] shipped, reproducible sample data
9	Test cases & results	[G9] evidence from real runs
10	API documentation	[G10] the three APIs the harness drives
11	Installation & evaluation	[G11] jury quick-start
12	Impact & roadmap	what this unlocks for MOSIP
13	Submission checklist	[G1-G12] compliance cross-reference

1 Executive summary

the pitch in 30 seconds - say this first, slides 1-4 of the deck

AutoConform is a standalone harness that turns OpenID Foundation conformance testing of MOSIP's Inji credential products into a single, unattended, CI-native command. It spins up the official **OpenID Foundation conformance suite** together with **Inji Certify 0.14** (OpenID4VCI 1.0 issuer) and **Inji Verify 0.18.2** (OpenID4VP / HAIP verifier) on one Docker network, drives the suite's REST API exactly as the official *run-test-plan.py* does, automatically completes the interactive WAITING handoffs that normally stall a test run, maps every suite module into **MOSIP's own TestNG / Extent reporting**, and fails the build when unexpected conformance regressions appear.

In one line: **conformance-as-code for the Inji credential ecosystem.**

826 lines of Python orchestration	505 lines of Java / TestNG bridge	328 lines of bash + PowerShell	602 lines of compose + configs	286 lines of docs
--	--	---	---	-----------------------------

Design principle - orchestration only

We implement **no OpenID protocol tests ourselves**. Protocol logic stays in the official OpenID Foundation suite; reporting stays in MOSIP's api-testrig conventions. AutoConform only orchestrates, maps, and gates. This keeps the harness authoritative forever - it can never disagree with the certification body about what the spec says.

2 The problem

why conformance testing hurts today

- **The suite is UI-driven.** Plans are created by clicking, variants are selected by hand, results live in a web dashboard. Nothing about that survives a git commit.
- **Interactive OpenID4VP modules stall.** The suite enters WAITING until a wallet completes a presentation - a human must paste links by hand.
- **Nothing re-checks compliance on every change.** Inji Certify and Inji Verify release frequently, but conformance is re-verified only when someone remembers to click through.
- **Results never reach MOSIP's reporting pipeline.** api-testrig / TestNG / Extent is what Inji teams review daily - conformance was invisible there.
- **Spec drift is found late.** Draft-era fields still sent against 1.0 endpoints surface as surprise failures during certification prep, when they are most expensive to fix.

Why it matters

Verifiable credentials only work if every issuer and verifier implements the spec identically - one non-conformant field breaks interoperability across the whole wallet ecosystem. MOSIP-based ID systems serve **500M+ people**; the Inji stack is their credential layer. A single manual conformance sweep takes hours of expert time per release. Compliance should be as automatic as unit tests.

3 The solution

what we built - four moving parts, one command

Layer	What it does
Compose stack	Official OIDF conformance suite (server, mongodb, nginx :8443) + Inji Certify 0.14 (certify, certify-nginx, postgres 15) + Inji Verify 0.18.2 (verify-service, verify-ui, postgres 13). Seven healthchecked services on one bridge network, pinned image tags.

Layer	What it does
Orchestration runner	Python wrapper around the suite REST API: create plan, start each module, poll to FINISHED, export HTML evidence. Renders committed plan templates with environment endpoints, supports parallel plans and modules, and can delegate to the official run-test-plan.py.
WAITING handoff	When the suite waits for a wallet, the runner acts as the wallet: it delivers credential offers to Certify (pre-authorized-code flow) and hands VP authorization requests to the suite's fake wallet (OpenID4VP). No human in the loop.
TestNG bridge	Java classes under io.mosip.testrig.apirig.openid convert results.json into the same TestNG / Extent reports MOSIP api-testrig publishes, with a benchmark gate that fails the build on unexpected failures.

Everything is invoked through one entry point: **run-conformance.sh** (bash / WSL / Linux / macOS) or **run-conformance.ps1** (Windows PowerShell). Modes: **--component certify**, **--component verify**, **--combined** (plus **--parallel** to run both plans concurrently). Each path: compose up, wait healthy, create/run/poll plans, write results.json, produce TestNG/Extent reports, fail on benchmark regression.

4 Architecture

system design and data flow - also the [G6] architecture diagram deliverable

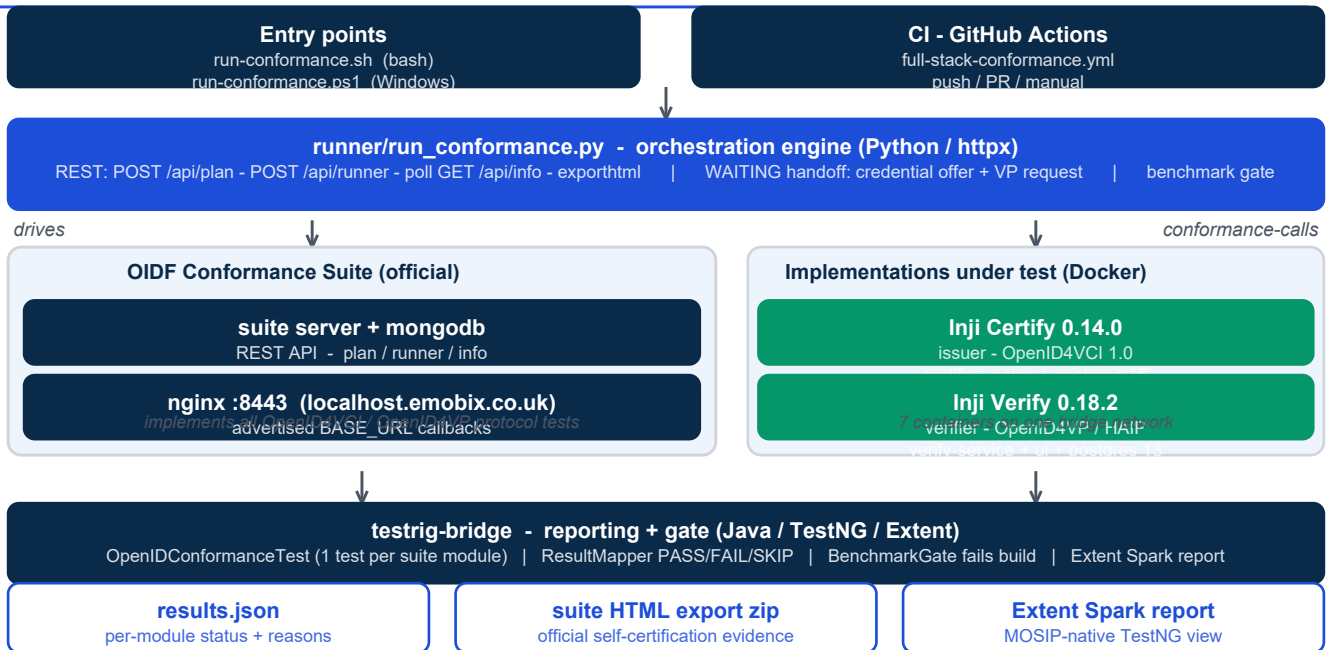


Figure 1 - AutoConform system architecture. Solid arrows are synchronous orchestration calls; the suite container itself makes the conformance calls to the implementations.

Three clean layers. Infrastructure (compose), orchestration (Python), reporting (Java/TestNG). The suite's `nginx` service is aliased `localhost.emobix.co.uk` so the suite's advertised `BASE_URL` callbacks resolve on the bridge network; plan JSON uses Docker DNS names (`http://certify-nginx`, `http://verify-service:8080/v1/verify`) because the suite process - not the host - calls the implementations. `Extra_hosts` entries bridge the containers back to the suite hostname.

OIDF suite	Python runner	TestNG bridge	GitHub Actions
protocol source of truth	REST orchestration	MOSIP reporting	CI gate on push/PR

5 Technology stack

and why each layer was chosen

Layer	Technology	Why it was chosen
Orchestration engine	Python 3.10+ / httpx (826 LOC)	Thin, auditable REST wrapper; mirrors official <code>run-test-plan.py</code> flow; async-friendly parallelism
Reporting bridge	Java 11 + Maven, TestNG + Extent (505 LOC)	Same reports MOSIP api-testrig already publishes - zero retraining for Inji teams

Layer	Technology	Why it was chosen
Infrastructure	Docker Compose - 7 services, pinned images	Certify 0.14.0, Verify 0.18.2, Mongo 6.0.13, Postgres 15/13, nginx - fully reproducible
Conformance suite	OpenID Foundation prebuilt image	The authoritative implementation of every OpenID4VCI / OpenID4VP test module
Entry points	Bash + PowerShell (328 LOC)	One command per platform; --component / --combined / --parallel / --skip-compose flags
CI	GitHub Actions (full-stack-conformance.yml)	compose up, conformance, TestNG bridge, artifact upload; 120-minute budget; runs on push and PR
Config as data	JSON plans / variants (9 issuer + 2 verifier)	Edit JSON, never click the suite UI - wrong variants look like product bugs

Reproducibility discipline

Every image tag is pinned in .env.example, every service has a healthcheck with a start period, and every plan template is committed. Anyone - including the jury - re-runs the exact same evaluation bit-for-bit. The GitHub Actions workflow provisions the identical stack on ubuntu-latest with a single checkout.

6 Key features

the three things that make this more than a script

6.1 The orchestration engine

- Mirrors the documented REST flow: wait GET /api/plan?length=1 ready, POST /api/plan, GET /api/plan/{id}, POST /api/runner per module, poll GET /api/info/{testId}, exporthtml package.
- **--parallel** runs Certify and Verify plans concurrently (2 threads) and suite modules concurrently (capped at 4) - dramatic wall-clock reduction on combined runs.
- Renders committed 1.0 plan templates with environment endpoints, so the same repository targets local Docker or a deployed environment (ENV_ENDPOINT - same contract as api-testrig).
- Optional escape hatch: --official-script delegates to the OIDF's run-test-plan.py when a byte-for-byte official run is required.
- Every module result carries provenance: testId, raw suite status, mapped PASS/FAIL/SKIP, expected flag, and reason.

6.2 Automated WAITING handoff - the innovation

This is the single hardest problem in OpenID4VP/OpenID4VCI testing - most teams demo it with a human holding a phone. AutoConform makes the runner **act as the wallet**:

Issuer side. On WAITING, the runner POSTs farmer identity claims (id, fullName, mobile, DOB, gender, farmerID) plus a transaction code to Certify's **/pre-authorized-data** API, resolves the credential-offer URI, fetches the offer JSON, and delivers it by value to the suite's offer endpoint (suite 5.31 rejects http:// offer URIs - we route around that).

Verifier side. The runner creates a Verify **vp-request** with an SD-JWT presentation definition (DCQL constraints on vct), rewrites the request to the suite's authorize endpoint, and completes the presentation against the fake wallet - unattended.

Modules that genuinely cannot run unattended are **explicitly skipped via config** (expected-skips.json) - never silently green, never hidden.

6.3 Honest results governance

Known product gaps become **data, not noise**. runner/configs/expected-failures.json documents 18 module failures with root-cause comments:

- Expected failures surface as TestNG skips with reasons - visible, never silently green.
- **benchmark.json gate**: fails the build on unexpected failures; optional minimum pass rate, required-module list, and unexpected-skip detection.
- **result_diff.py**: compare any run against a baseline - instantly see new failures and new passes after a product upgrade.
- Certification discipline: expected-failures must be empty for a certification candidate; known issues belong in a formal waiver, not a silent skip.

7 Working prototype & demo [G2]

what a live run looks like - also the storyboard for the [G4] video demo

The repository is a **functional prototype**, not a mock. The following run was exercised against the pinned Docker images and takes roughly four minutes end-to-end:

#	Step	What happens
1	Stack up	run-conformance.sh --component certify starts compose; healthchecks bring the suite, Certify and its database to ready; the runner waits for the suite REST API.
2	Plan created	The runner POSTs the oid4vci-1_0-issuer-test-plan with committed variants; the suite enumerates its modules.
3	The wow moment	A module enters WAITING. The runner calls Certify's pre-authorized-data API with farmer claims, fetches the credential offer, and delivers it to the suite - the module resumes and PASSES with no human.
4	Verifier mirrored	For Verify, the runner creates a VP request, rewrites it to the suite's authorize endpoint, and hands it to the fake wallet the same way.
5	Evidence	Console prints the summary JSON; the suite's official HTML export zip and the Extent Spark report are written; the benchmark gate evaluates.
6	CI proof	The same run is green on GitHub Actions, with results.json, diff.json and the HTML export zips uploaded as artifacts on every push and PR.

For the video demo [G4]

Record exactly the six steps above as a 2-4 minute screen capture: terminal on the left, Extent Spark report on the right. The WAITING handoff moment (step 3) is the shot that differentiates the project - give it 30 seconds of full-screen focus.

8 Test data [G8]

shipped, reproducible sample data - all in-repo

Artefact	Location	Content
Identity claims	farmer_identity_data.csv + FARMER_CLAIMS in run_conformance.py	id F001, fullName Ada Farmer, mobile 9999999999, DOB 1990-01-01, gender Female, farmerID FAR-001
Credential type	compose/certify/config/certify-*.properties	FarmerCredential (SD-JWT VC) as the issued credential configuration
Authorization	static transaction code	tx_code 1234 for pre-authorized-code flows
Presentation definition	SD_JWT_PRESENTATION_DEFINITION in run_conformance.py	DCQL input descriptor on vct = FarmerCredential; sd-jwt_alg_values = ES256
Verifier trust anchor	verifier-plan.json	Pinned request_object_trust_anchor_pem (EC P-256 suite root certificate)
Client identities	issuer-plan.json / verifier-plan.json	inji-certify-conformance (and -2) with scope mock_identity_vc_ldp; inji-verify-conformance
Environment	compose/.env.example	Pinned image tags and ports for the entire 7-service stack

No external accounts, paid APIs, or seeded services are required - every credential, claim and key needed to reproduce the evaluation is committed to the repository.

9 Test cases & results [G9]

evidence from real runs against the pinned images

The OpenID Foundation suite itself enumerates the test cases - including negative-path edge cases such as invalid KB-JWT signature, wrong nonce/aud, iat in past/future, invalid SD hash, invalid credential signature, unknown credential configuration/identifier, access token in query, and missing proofs. AutoConform's evidence layer makes those results portable and gated:

Evidence layer	What it provides
Per-module evidence	results.json records testId, raw suite status, mapped result, expected-failure flag and human-readable reason for every module.
Two reporting views	The suite's official HTML export zip (certification-grade evidence) plus an Extent Spark report (the MOSIP-native TestNG view teams already review).
Benchmark gate	benchmark.json fails the build on unexpected failures (failOnUnexpectedFailure = true); pass-rate floor and required modules are configurable.
Run-over-run diff	result_diff.py compares the current run with results/previous.json and emits diff.json - new failures, new passes, unchanged.
Expected failures	18 modules documented with root causes (7 Certify draft-13 vs 1.0 gaps; 11 Verify draft-21 vs HAIP-Final gaps) - visible as explained skips, not mystery reds.
Expected skips	6 interactive modules not startable unattended are listed with a comment explaining exactly why.

What the numbers say

7 Certify failures share one root cause (credential_configuration_id vs draft-13 fields); 11 Verify failures share one root cause (OpenID4VP draft 21 vs HAIP 1.0 Final). In other words: **two precise, fixable spec-drift findings** - exactly the class of issue this harness exists to surface, and each one comes with the upgrade path that turns it green.

10 API documentation [G10]

the three APIs the harness drives

10.1 OpenID Foundation conformance suite REST API

Method	Endpoint	Purpose / notes
GET	/api/plan?length=1	Readiness probe - 200 means suite + Mongo are up. Local dev profile needs no token.
POST	/api/plan?planName=...&variant;={...}	Create a plan. Body = plan configuration JSON; variant = JSON object of profile switches.
GET	/api/plan/{id}	List the plan's modules.
POST	/api/runner?test=MODULE&plan;=PLAN_ID &variant;={...}	Start one module. Returns the testId.
GET	/api/info/{testId}	Poll module status until FINISHED / INTERRUPTED; exposes "exposed" values used for the WAITING handoff.
GET	/api/plan/exporthtml/{planId}	Download the human-readable HTML evidence package (zip).

10.2 Inji Certify - credential issuance

Method	Endpoint	Request / response
POST	/v1/certify/pre-authorized-data	Creates pre-authorized issuance data. Request: { credential_configuration_id, claims, expires_in, tx_code }. Response wraps { credentialOfferUri }; the offer JSON is then fetched and delivered to the suite by value.
GET	/.well-known/openid-credential-issuer	Issuer metadata discovery (healthchecked via certify-nginx).

10.3 Inji Verify - presentation request

Method	Endpoint	Request / response
POST	/v1/verify/vp-request	Creates an OpenID4VP request. Request: { clientId, presentationDefinition }. Response wraps { requestId, requestUri }; the runner rewrites requestUri onto the suite's authorize endpoint.
GET	/v1/verify/actuator/health	Service healthcheck.

All three clients live in runner/oidf_client.py and runner/run_conformance.py with error surfaces (HTTP status + truncated body) printed for every failure - the same documentation the docs/api-automation.md page expands on.

11 Installation & evaluation [G11]

jury quick-start - works on Windows, macOS and Linux

Prerequisites

- Docker + Docker Compose (Docker Desktop on Windows/macOS).
- Python 3.10+ with pip (runner/requirements.txt: httpx).
- Java 11+ and Maven (only for the TestNG/Extent bridge step).
- One-time hosts entry: 127.0.0.1 localhost.emobix.co.uk



What you will see

- Compose healthchecks going green, then "Conformance suite ready".
- Module-by-module progress, then a summary JSON with passed / failed / skipped and benchmarkMet.
- results/<mode>/results.json, the suite HTML export zip, and the Extent Spark report.
- Exit code reflects the benchmark gate - CI fails if unexpected failures remain.

Useful variations



Full troubleshooting (hosts file on Windows, 400 on plan create, Postgres volume resets, TestNG finding no modules) is documented in docs/troubleshooting.md.

12 Impact & roadmap

what this unlocks for MOSIP

Impact now

- Hours of expert manual testing become **one command in CI**, on every push and PR.

- Spec drift is caught at commit time with **root-cause comments**, not mystery failures.
- Certification evidence (HTML packages + exact variant matrix) is generated automatically.
- Inji teams keep their existing TestNG/Extent workflow - **zero retraining**.

Next

#	Next step
1	Wallet stub service to automate the remaining interactive OpenID4VP modules - remove the last skips.
2	Certify 1.0 field upgrade - flip 7 expected failures to real passes when credential_configuration_id support lands.
3	HAIP 1.0 Final profile support in the Verify pipeline (request_uri_signed, x509_hash) - the other 11.
4	Trend dashboard: pass-rate history per release across result_diff.json artifacts.
5	OpenID official certification submission for both components via the hosted environment.

Target audience: MOSIP platform team, Inji module maintainers, the OpenID Foundation certification community, and any government deploying wallet-based credential ecosystems on MOSIP - the harness serves every one of them the same way: run it, read the report, ship.

13 Submission checklist [G1-G12]

compliance cross-reference against the official MOSIP Decode 2026 submission guidelines

ID	Guideline item	Where it is satisfied	Status
G1	Source code MANDATORY	Well-organised, commented repo: compose / runner / testrig-bridge / scripts / docs; docstrings and Javadoc throughout; ~2.5k LOC. Push to GitHub with all files needed to run.	Yes
G2	Working prototype / demo MANDATORY	Functional one-command harness; live run storyboard in section 7; CI green on push/PR.	Yes
G3	Pitch presentation / slide deck MANDATORY	This document is deck-ready: sections map 1:1 to slides; speaker-note content embedded in each section's subtitle.	Yes
G4	Video demo (2-5 min) RECOMMENDED	Record section 7's six steps (screen capture + Extent report). Upload link on the closing slide.	Team action
G5	Design mockups / wireframes OPTIONAL	CLI-centric project: include the Figure-1 architecture render, a CI pipeline flow sketch, and a WAITING-handoff sequence diagram (can be hand-drawn).	Team action
G6	Architecture diagram RECOMMENDED	Figure 1 in section 4 (render/export to PNG for the deck); docs/architecture.md details components, data flow and interactions.	Yes
G7	Test data RECOMMENDED	All sample data committed: farmer identity CSV, FarmerCredential config, tx_code, DCQL presentation definition, verifier trust anchor, .env.example - see section 8.	Yes
G8	Test cases & results RECOMMENDED	Per-module results.json with reasons, Extent report, diff.json, benchmark gate output, expected failures/skips with root causes - see section 9. Attach artifacts.	Yes
G9	Usage & installation instructions MANDATORY	README quick-start, section 11 quick-start (Windows + unix), docs/troubleshooting.md.	Yes
G10	API documentation IF APPLICABLE	All three driven APIs documented with methods, endpoints, request/response formats and examples - section 10 and docs/api-automation.md.	Yes
G11	Git repository with version-control history MANDATORY	Ensure the pushed GitHub repository retains feature-branch history showing team collaboration and project progression.	Team action
G12	College / university credentials MANDATORY	Each member uploads a valid student ID as proof of enrolment (platform step, not repository).	Team action

Legend + remaining team actions

MANDATORY items are satisfied in the repository today. The four *team actions* (record video, simple diagrams, verify git history on GitHub, upload student IDs) are platform or media steps the repository cannot perform on the team's behalf - everything else ships with the code. **G-numbers are internal cross-references** used throughout this document.

AutoConform - conformance as code for the Inji credential ecosystem. Repository: `Testing_for_inji_certify_and_inji-verify-main`.
Prepared for MOSIP Decode 2026.